A First-Order Bundle Method for Smoothing the Pareto Front of Non-convex Multi-objective Optimization

Paul Grigas
University of California, Berkeley
Berkeley, CA 94720
pgrigas@berkeley.edu

Jiahui (Jeffrey) Cheng
University of California, Berkeley
Berkeley, CA 94720
jeffrey_cheng@berkeley.edu

Cheng Zeng
University of California, Berkeley
Berkeley, CA 94720
cheng_zeng@berkeley.edu

Rongchuan Shi
University of California, Berkeley
Berkeley, CA 94720
rongchuan@berkeley.edu

Hua Ye
University of California, Berkeley
Berkeley, CA 94720
hua01@berkeley.edu

Abstract

Multi-objective optimization (MOO) requires balancing multiple potentially conflicting criteria. It finds applications where the goal is to generate solutions that represent diverse user preferences. A traditional practice to solve MOO is linear scalarization (LS) via a weight vector. However, in reality, the right weights are rarely known at prior to the decision maker. Existing approaches solve the MOO problem at one single weight. We propose a first-order bundle method that systematically reuses zero-th and first-order information of the objective functions, in order to generate ϵ-solutions for all weight vectors. Theoretically, our method is comparable to the uniform discretization approach as the baseline in terms of oracle complexity. Empirically, we demonstrate the advantage of our method over the baseline on structured problems, including bandits, multi-objective-gymnasium, and multi-objective LLM alignment. The code is available at `https://github.com/MarshmallowJeffrey/First-order-method-for-smooth-MOO`.

1 Introduction

Many problems in machine learning involve the simultaneous optimization of multiple objective functions. In physics-informed neural networks (PINNs) training, multiple loss terms induced by PDE residuals and boundary or initial conditions must be optimized simultaneously [YBHNL26]. In multi-task learning, a shared model must perform well across several tasks at once, each with its own loss function [R97]. In multi-class classification, the per-class losses define a collection of objectives that must be jointly minimized to achieve good predictive performance across all classes [Bis06]. These settings—and others such as hyperparameter optimization, federated learning, and fairness-aware training—give rise to the *multi-objective optimization* (MOO) problem.

Preprint.

Formally, let θ denote the decision variable in the feasible set $\Theta \subseteq \mathbb{R}^d$, MOO seeks to minimize K objectives $F_k : \mathbb{R}^d \to \mathbb{R}$:

$$\min_{\theta \in \Theta} \{F(\theta) := [F_1(\theta), \ldots, F_K(\theta)]^T\}.$$

We consider optimizing the above vector-valued objective to Pareto optimality. A solution θ^* is *Pareto optimal* if no feasible point improves one objective without hurting another objective. The set of all Pareto-optimal solutions is the *Pareto set* Θ_{PF}, and its image $\mathcal{F}_{\text{PF}}$ under F is the Pareto front (PF). Formal definitions are provided in Section 2.

A common approach to solve MOO is via *linear scalarization*. For example, in the bi-objective case ($K = 2$), given $\lambda \in \Delta_2$, linear scalarization (LS) solves

$$\theta^*_\lambda \in \arg\min_{\theta \in \Theta} \{F_\lambda(\theta) := \lambda_1 F_1(\theta) + \lambda_2 F_2(\theta)\}.$$

For a single, fixed λ, this reduces to a scalar optimization problem. However, for many machine learning and operations research applications, the weights need to be assigned at prior. One potential resolution is to find good solutions for *all* weight vectors $\lambda \in \Delta_2$—that is, to approximate the entire Pareto front. The standard approach is uniform discretization, which partitions Δ_2 into a uniform grid and solves the multi-objective optimization problem at each grid point. Solutions to the MOO problem at other grid points are obtained by rounding to the closest grid point. However, as K scales up, the computation time also scales up exponentially, which makes the uniform discretization approach intractable. This motivates the question:

Is there an **efficient** alternative to solve the MOO problem **for all** weight vectors in the unit simplex?

We answer this by proposing a *first-order bundle method*. The key idea is to maintain a *bundle*—a collection of zeroth-order and first-order information of the objective functions—and to construct from this bundle a *progress criterion* that certifies solution quality *uniformly* over all grid points in the unit simplex.

1.1 Our Contributions

- **C1) A bundle algorithm with convergence guarantee.** In Section 4, we propose a first-order bundle method (Algorithm 2), which adaptively selects a weight vector λ and solves the MOO problem for all weight vectors in the unit simplex for general K objective functions. Theorem 1 shows that the uniform error tolerance converges to ϵ in oracle complexity $O((\frac{1}{\epsilon})^K)$.
- **C2) Empirical verification.** In section 5, we present numerical results on diverse tasks, including structured bandits, MO-Gymnasium benchmarks, and LLM alignment tasks. Our bundle method outperforms the uniform discretization method for large K under fixed total computation budgets.

1.2 Related Work

Existing approaches address this challenge from different angles. Methods such as the Multiple Gradient Descent Algorithm (MGDA) [Dés12] and its variants [SK18, LLJ$^+$21, NSA$^+$22] seek a single Pareto-stationary point by combining task gradients, but they do not characterize the full Pareto front. Grid-based scalarization methods [SK18] discretize Δ_K and solve each subproblem independently, but this is wasteful: gradient information computed at one weight vector is discarded when solving for another. Neither approach provides a principled mechanism for *reusing* zeroth-order and first-order information across different scalarization weights.

Notation. For any positive integer m, let $[m]$ denote the set $[m]$. Let $\Delta_K := \{\lambda \in \mathbb{R}^K : \sum_{k=1}^K \lambda_k = 1, \lambda \geq 0\}$ be the unit simplex of dimension K. Let $\nabla F(\theta)$ be the gradient of function F at point $\theta \in \mathbb{R}^d$. We will make use of a generic given norm $\|\cdot\|$ on $w \in \mathbb{R}^d$.

2 Model Setup and Preliminaries

We first provide definitions for MOO, then discuss additional assumptions for our problem setup.

Definition 2.1 (Pareto optimality and Pareto front)**.** *A feasible point $\theta^* \in \Theta$ is Pareto optimal if there does not exist another point $\theta \in \Theta$ such that $F_k(\theta) \leq F_k(\theta^*)$ for all $k \in [K]$ and $F_t(\theta) < F_t(\theta^*)$ for some $t \in [K]$. The Pareto set (PS) is the collection of all Pareto-optimal solutions, denoted as Θ_{PF}, and its image under F is the Pareto front (PF), $\mathcal{F}_{PF} := \{F(\theta^*) : \theta^* \in \Theta_{PF}\}$.*

We now impose the following assumptions on each objective function F_K.

Assumption 2.1 (Smoothness)**.** *Assume each $F_k : \mathbb{R}^d \to \mathbb{R}$ is L_k-smooth with respect to the l_2-norm, i.e: $\|\nabla F_k(\theta_1) - \nabla F_k(\theta_2)\|_2 \leq L_k\|\theta_1 - \theta_2\|_2, \forall \theta_1, \theta_2 \in \Theta$.*

Assumption 2.2 (Boundedness)**.** *Assume each $F_k : \mathbb{R}^d \to \mathbb{R}$ is bounded below, i.e: $F_k^* := \inf_{\theta \in \mathbb{R}^d} F(\theta) > -\infty$. Note that finiteness of F_k^* is not a strong assumption in many applications that we care about; for example, in machine learning most loss functions are nonnegative.*

Assumption 2.3 (Non-convexity)**.** *Assume each $F_k : \mathbb{R}^d \to \mathbb{R}$ is non-convex. This is typical as modern machine learning objectives are highly non-convex.*

Our goal is to solve the linearly scalarized multi-objective optimization (MOO) problem:

$$\theta_\lambda^* \in \arg\min_{\theta \in \Theta} \{F_\lambda(\theta) := \sum_{k=1}^{K} \lambda_k F_k(\theta)\}, \tag{1}$$

for all weight vector $\lambda \in \Delta_K$. That is, to approximate the entire Pareto front.

Note that $F_\lambda(\cdot)$ is L_λ-smooth with $L_\lambda := \sum_{k=1}^K \lambda_k L_k$, bounded below with $F_\lambda^* := \inf_{\theta \in \mathbb{R}^d} F_\lambda(\theta) \geq \sum_{k=1}^K \lambda_k F_k^* > -\infty$, and non-convex.

We use an *approximate solution map* $\hat{\theta}(\cdot) : \Delta_K \to \mathbb{R}^d$ to track the set of solutions. To evaluate the quality of an approximate solution map in the non-convex setting, we might use the following measure of worst-case solution map stationarity:

$$\epsilon_{\mathrm{sm-stat}}(\hat{\theta}(\cdot)) := \sup_{\lambda \in \Delta_K} \|\nabla F_\lambda(\hat{\theta}(\lambda))\|^2. \tag{2}$$

Our goal will be to develop an algorithm, with corresponding complexity analysis, to find an approximate solution map $\hat{\theta}(\cdot)$ satisfying $\epsilon_{\mathrm{sm-stat}}(\hat{\theta}(\cdot)) < \epsilon$ for a given error tolerance $\epsilon > 0$.

3 Uniform Discretization — The Baseline Method

In this section, we describe the uniform discretization method, which serves as the baseline method for solving problem 1. The idea is to partition the unit simplex Δ_K into a uniform grid, run stochastic gradient descent (SGD) method at each grid point, and approximate the solution at any query $\lambda \in \Delta_K$ by rounding to the nearest grid point in L_1-norm distance.

Let $r \in \mathbb{Z}_+$ be a grid resolution parameter. Define the uniform simplex grid

$$\mathcal{G}_r \;:=\; \big\{\lambda \in \Delta_K \;:\; r\lambda \in \mathbb{Z}_+^K\big\}. \tag{3}$$

That is, $\mathcal{G}_r$ consists of all grid points whose coordinates are multiples of $1/r$. By stars-and-bars argument, $|\mathcal{G}_r| = \binom{r+K-1}{K-1}$, so the grid size grows exponentially in the number of objectives K. We enumerate $\mathcal{G}_r = \{\lambda^{(1)}, \ldots, \lambda^{(N)}\}$ in lexicographic order, which makes consecutive grid points ℓ_1-close ($\|\lambda^{(g+1)} - \lambda^{(g)}\|_1 \le 2/r$).

Solving along the grid. At each grid point $\lambda^{(g)}$ the method runs stochastic gradient descent (SGD) on $F_{\lambda^{(g)}}$ with the fixed step size $1/L_{\lambda^{(g)}}$, where $L_{\lambda^{(g)}} = \sum_{k=1}^K \lambda_k^{(g)} L_k$.

Query and Grid point bundle construction. The method returns the table $\{(\lambda^{(g)}, \widehat{\theta}^{(g)})\}_{g=1}^N$ of grid points and their final iterates. For any query $\lambda \in \Delta_K$, the approximate solution map rounds to the nearest grid point in ℓ_1 distance,

$$\widehat{\theta}(\lambda) \;:=\; \widehat{\theta}^{(g^*)}, \qquad g^* \;=\; \operatorname*{arg\,min}_{g \in [N]} \|\lambda - \lambda^{(g)}\|_1. \tag{4}$$

The collection $\mathcal{B}_m := \{\widehat{\theta}^{(g)}\}_{g=1}^N$ of final iterates forms the baseline's bundle, scored by the same worst-case stationarity measure GN^* used for Algorithm 2.

3.1 The Uniform Discretization Algorithm

Algorithm 1 Uniform Discretization

Input: objectives $F_1, \ldots, F_K$ with smoothness constants $L \in \mathbb{R}^K$; initial point $\theta_0 \in \mathbb{R}^d$; resolution r; passes P; steps per point S.
Enumerate $\mathcal{G}_r = \{\lambda^{(1)}, \ldots, \lambda^{(N)}\}$ in lexicographic order, $N = \binom{r+K-1}{K-1}$.
Initialize $\widehat{\theta}^{(g)} \leftarrow \theta_0$ for all $g \in [N]$.
for pass $= 1, \ldots, P$ **do**
  $x \leftarrow \widehat{\theta}^{(1)}$
  for $g = 1, \ldots, N$ **do**
    $x \leftarrow \widehat{\theta}^{(g)}$ **if** pass > 1 {else keep chained x from previous grid point}
    $L_\lambda \leftarrow \sum_{k=1}^K \lambda_k^{(g)} L_k$
    for S steps **do**
      $x \leftarrow x - \frac{1}{L_\lambda} \nabla F_{\lambda^{(g)}}(x)$
    end for
    $\widehat{\theta}^{(g)} \leftarrow x$
  end for
end for
return $\big\{(\lambda^{(g)}, \widehat{\theta}^{(g)})\big\}_{g=1}^N$.

4 An Adaptive Algorithm for Smoothing the Pareto Front

We propose an algorithmic framework for solving the MOO problem (1) based on a bundle of zeroth and first-order information as well as associated progress criteria built from these bundles. The key objects in play are:

1. A bundle of zeroth and first-order information, computed at a given list of points $\theta_1, \ldots, \theta_m$, denoted by
$$\mathcal{B}_m := \{(\theta_i, F_1(\theta_i), \ldots, F_K(\theta_i), \nabla F_1(\theta_i), \ldots, \nabla F_K(\theta_i))\}_{i=1}^m.$$
2. A mapping $T(\cdot; \mathcal{B}_m) : \Delta_K \to \mathbb{R}^d$, associated with the bundle $\mathcal{B}_m$, where the output $T(\lambda; \mathcal{B}_m)$ is "easily computable" given the information in the bundle $\mathcal{B}_m$.
3. A "gradient norm" function $\mathrm{GN}(\cdot; \mathcal{B}_m) : \Delta_K \to \mathbb{R}$ associated with the bundle $\mathcal{B}_m$, e.g:
$$\mathrm{GN}(\lambda; \mathcal{B}_m) = \min_{\theta_i \in \mathcal{B}_m} \{\|\nabla F_\lambda(\theta_i)\|^2\}.$$
4. A family of constants $\tilde{L}_\lambda > 0$ for $\lambda \in \Delta_K$.

Example 4.1 (Vanilla gradient descent)**.** *For a fixed $\lambda \in \Delta_K$, define a mapping $T(\cdot; \mathcal{B}_m)$ by*

$$T(\lambda; \mathcal{B}_m) := \theta_{i^*} - \frac{1}{L_\lambda} \nabla F_\lambda(\theta_{i^*}), \text{ for } i^* \in \arg\min_{\theta_i \in \mathcal{B}_m} \left\{ F_\lambda(\theta_i) - \frac{1}{2L_\lambda} \|\nabla F_\lambda(\theta_i)\|^2 \right\}, \quad (5)$$

with ties broken according to a least index rule (for example).

For a given bundle $\mathcal{B}_m$ of zeroth and first-order information computed at points $\theta_1, \ldots, \theta_m$ and a given $\bar{\theta} \in \mathbb{R}^d$, we denote

$$\mathcal{B}_{m+1} := \mathcal{B}_m \cup (\bar{\theta}, F_1(\bar{\theta}), \ldots, F_K(\bar{\theta}), \nabla F_1(\bar{\theta}), \ldots, \nabla F_K(\bar{\theta}))$$

as the new bundle with additional information computed at $\bar{\theta}$. *Note that whenever a new point is added to the bundle, function values and gradients are assumed to be calculated at this new point.*

4.1 An Adaptive algorithm with BundleUpdate

In this subsection, we present an adaptive algorithm to solve the MOO problem (1) for the gradient norm progress criterion function.

Let's consider the following algorithm. We use a subroutine to iterate adding points to the bundle at a fixed $\lambda \in \Delta_K$. Specifically, let $\mathcal{B}_0$ be a given initial non-empty bundle, and let $\mathrm{BundleUpdate}_M(\lambda; \mathcal{B}_0)$ denote the bundle that results after applying $T(\lambda; \cdot)$ for M steps starting with $\mathcal{B}_0$. In other words, we add $\bar{\theta}_1 := T(\lambda; \mathcal{B}_0)$ to create bundle $\mathcal{B}_1$, then $\bar{\theta}_2 := T(\lambda; \mathcal{B}_1)$ to create bundle $\mathcal{B}_2$, etc. until adding M total points to the bundle resulting in the final bundle $\mathcal{B}_M = \mathrm{BundleUpdate}_M(\lambda; \mathcal{B}_0)$.

The adaptive algorithm under non-convexity became

Algorithm 2 Adaptive Algorithm for non-convex function

Input: Number of classes K; max outer iterations T; initial point θ_0; final accuracy parameter ϵ.
Initialize: Initial bundle $\mathcal{B}_0$ built at θ_0; Anchor $\mathcal{A}_0 = \{(\text{centroid}(\Delta_K), \theta_0)\}$.
for iteration $t = 0, 1, \ldots, T-1$ **do**
  Compute $\lambda_t \in \Delta_K$ that maximizes $\min_{\theta_i \in \mathcal{B}_m} \|\nabla F_{\lambda_t}(\theta_i)\|^2$ and set $\mathrm{GN}_t^* := \mathrm{GN}(\lambda_t; \mathcal{B}_t)$ as the maximum value.
  if $\mathrm{GN}_t^* < \frac{2}{3}\epsilon$ **then**
    return anchor $\mathcal{A}_t$ to build the approximate solution path.
  else
    Set $\mathcal{B}_{t+1} = \mathrm{BundleUpdate}_{M_t}(\lambda_t; \mathcal{B}_t)$ where $M_t := \arg\min_{M_t} \{\mathrm{GN}(\lambda_t; \mathcal{B}_{t+1}) < \frac{\epsilon}{3}\}$
    Construct $\mathcal{A}_{t+1}$ from $\mathcal{A}_t$ and $(\lambda_t, \mathcal{B}_{t+1} \backslash \mathcal{B}_t)$
  end if
end for

4.1.1 Algorithm complexity analysis

Proposition 1. *For a bundle $\mathcal{B}_m$ generated by a sequence of points $\{\bar{\theta}_i := T(\lambda_i; \mathcal{B}_i) : \theta_i \in \mathcal{B}_m\}$ and any fixed $\lambda \in \Delta_K$, there exists a universal constant $C_\lambda > 0$ such that*

$$\mathrm{GN}(\lambda; \mathcal{B}_m) \leq \frac{C_\lambda L_\lambda}{m}.$$

Corollary 4.1. *For any given initial bundle $\mathcal{B}_0$ and any fixed $\lambda \in \Delta_K$, for some universal constant $C_\lambda > 0$, setting $\mathcal{B}_M = \mathrm{BundleUpdate}_M(\lambda; \mathcal{B}_0)$ for $M = \lceil \frac{C_\lambda L_\lambda}{\epsilon} \rceil$ yields*

$$\mathrm{GN}(\lambda; \mathcal{B}_M) < \epsilon.$$

Now, we have the following theorem to analyze Algorithm 2.

Theorem 1. *Let $\hat{\theta}(\cdot)$ be the approximate solution path built from the returned anchor $\mathcal{A}$. For each $\lambda \in \Delta_K$, let C_λ be the universal constant in Corollary 4.1. Define*

$$\mathrm{LipGN}(\lambda; \mathcal{B}_m) := \max_{\theta_i \in \mathcal{B}_m} \{\|J_F(\theta_i) J_F(\theta_i)^T\|_{1,\infty}\};\ E := 3 \sup_{\lambda \in \Delta_K} C_\lambda L_\lambda.$$

Then for some universal constant $C > 0$, after at most $\left(\frac{C\mathrm{LipGN}}{\epsilon}\right)^{K-1}$ outer iterations and at most $\left(\frac{C\mathrm{LipGN}}{\epsilon}\right)^{K-1} (\frac{E}{\epsilon})$ total oracle calls, it holds that $\mathrm{GN}_t^* \leq \epsilon$. In addition, $\epsilon_{\mathrm{sm-stat}}(\hat{\theta}(\cdot)) \leq \epsilon$.

Proof. See Appendix A.2 □

4.1.2 Using SGD in the inner loop

5 Numerical Experiments

In this section, we empirically verify the effectiveness of our first-order bundle algorithm 2 against the uniform discretization method 1 as the baseline. We present computational results for various tasks, including offline bandit, MO-Gymnasium, and LLM fine-tuning.

Worst-case error. In both experiments we compute a bundle $\mathcal{B}_m$: for the uniform-discretisation baseline (Algorithm 1), $\mathcal{B}_m$ is the set of *last iterates* obtained by running gradient descent on F_λ at each node λ of the coarse grid of resolution r — a fixed set of $\binom{r+K-1}{K-1}$ points spread uniformly over Δ_K; for Algorithm 2, $\mathcal{B}_m$ is the set of points the algorithm accumulates, allocated adaptively to the weights where the gradient norm progress criterion is largest.

We then measure first-order *stationarity* of the scalarised objective along the solution map, reporting the worst-case squared gradient norm

$$\epsilon_{\text{sm-stat}}\big(\widehat{\theta}(\cdot)\big) := \sup_{\lambda\in\Delta_K} \big\|\nabla F_\lambda\big(\widehat{\theta}(\lambda)\big)\big\|^2 = \sup_{\lambda\in\Delta_K} \min_{\theta_i\in\mathcal{B}_m} \|\nabla F_\lambda(\theta_i)\|^2 = \sup_{\lambda\in\Delta_K} \mathrm{GN}(\lambda;\mathcal{B}_m) \quad (6)$$

A small value of (6) certifies that $\widehat{\theta}(\lambda)$ is approximately stationary for F_λ *uniformly* over the simplex Δ_K.

Two budget axes. We benchmark each method along two complementary axes:

1. **Gradient evaluations.** We count one gradient evaluation per call to a single $\nabla F_k(\cdot)$, so each scalarised gradient-descent step costs K evaluations. For applications where the gradient is expensive — for instance, large-scale neural-network training where each ∇F_k is a backpropagation pass — this axis directly reflects wall-clock cost.
2. **CPU time.** We count the total CPU time for both methods.

Checkpoint cadence. For each method, we record (gradient evaluations, CPU time, worst-case error) tuples at periodic checkpoints, triggered when M gradient evaluations have accumulated since the last checkpoint.

Implementation details of Algorithm 2. For Algorithm 2, we make the following implementation choice that materially affect runtime cost:

- **Inner-loop pruning.** For weight λ_t at iteration t, we add only the point of the last iterate corresponding to the minimum gradient norm to the current bundle $\mathcal{B}_t$.

5.1 Synthetic Experiment

For generic non-convexity, we should decide on the neural network hyperparameter choices:

- NN architecture: how many hidden layers/nodes, CNN, MLP?
- Learning rate/GD step size?
- Activation function choice?
- Number of epochs?
- What optimizer? SGD? ADAM? Mini-batch SGD? what mini-batch size? What momentum coefficient?
- Intentionally add noise to each gradient evaluation?
- Xavier/He initialization for weight?

For simplicity, we consider a 1-hidden-layer MLP. In particular, let $W_1 \in \mathbb{R}^{h\times p}$ be the input-to-hidden weights, $b_1 \in \mathbb{R}^h$ be the hidden biases, $W_2 \in \mathbb{R}^{K\times h}$ be the hidden-to-output weights, $b_2 \in \mathbb{R}^K$ be the output biases. Let σ be the activation function, e.g: identity, ReLU.

The forward pass is

$$z := W_2\sigma(W_1x + b_1) + b_2.$$

5.1.1 Data generation

We sample synthetic data from a linear–softmax planted model. Let p be the feature dimension and n be the sample size. We draw a ground-truth weight matrix and a feature matrix

$$W_\star \in \mathbb{R}^{K\times p}, \quad (W_\star)_{i\ell} \overset{\text{iid}}{\sim} \mathcal{U}[-1,1], \qquad x_j \overset{\text{iid}}{\sim} \mathcal{N}(0,\, I_p), \quad j = 1,\dots,n,$$

and assemble $X = [\theta_1 \cdots x_n]^\top \in \mathbb{R}^{n\times p}$. Conditional on X, the labels are drawn from the categorical distribution induced by the planted linear softmax,

$$y_j \mid x_j \;\sim\; \text{Categorical}\big(\text{softmax}(W_\star x_j)\big), \qquad j = 1,\dots,n,$$

implemented numerically by computing the row-stochastic matrix $\Pi = \text{softmax}(XW_\star^\top) \in \mathbb{R}^{n\times K}$ and inverse-CDF-sampling each y_j from the j-th row of Π.

5.1.2 Experiment Plots

The following figures compares the performance of the adaptive bundle method with the uniform discretization approach as the baseline under various hyperparameter settings:

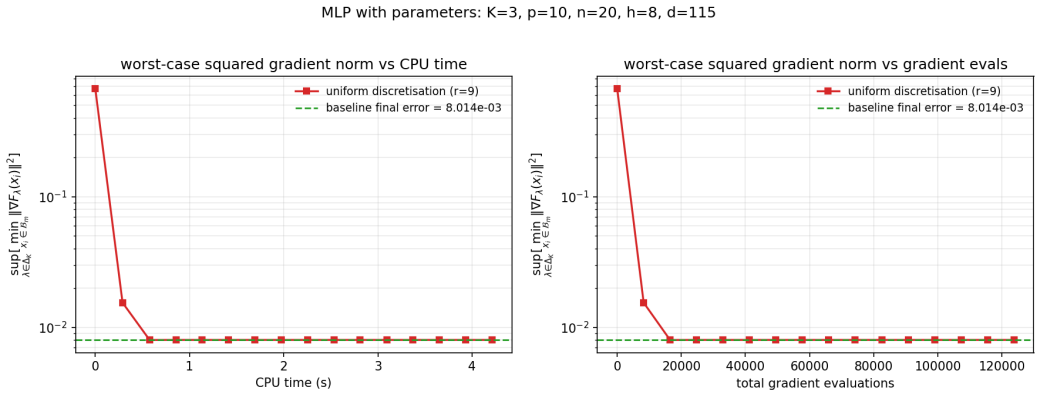

Figure 1: Worst-case squared gradient norm V.S. CPU time and grad evals for error tolerance 1e-2.

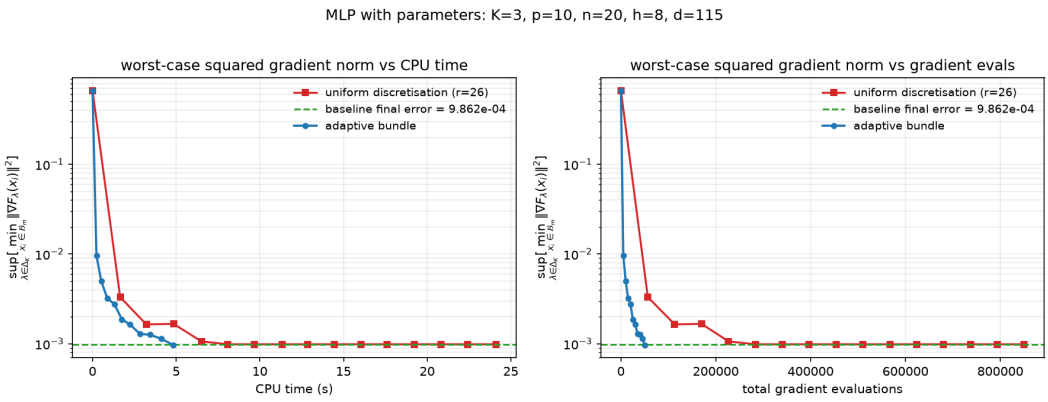

Figure 2: Worst-case squared gradient norm V.S. CPU time and grad evals for error tolerance 1e-3.

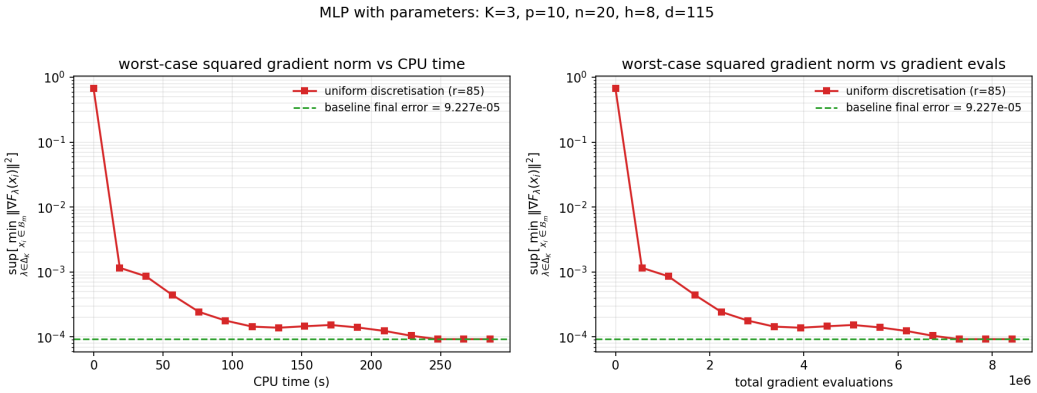

Figure 3: Worst-case squared gradient norm V.S. CPU time and grad evals for error tolerance 1e-4.

5.2 Toy example: Offline bandit

- Setting:
  - A single-state MDP with $|S| = 1$ and a finite action set $\mathcal{A}$.
  - $x_a \in [0, 1]$ is evenly spaced over $[0, 1]$ for each action $a \in \mathcal{A}$.
  - True mean rewards $R_1(a) = x_a, R_2(a) = 1 - x_a^4, R_0 = \log \pi_{\text{ref}}$ for $\pi_{\text{ref}} = \frac{1}{|\mathcal{A}|}$
  - A fixed balanced offline dataset $\mathcal{D} = \{(a_t, r_{1,t}, r_{2,t})\}_{t=1}^{T}$ where every action appears roughly equally often and rewards are observed with Gaussian noise with standard deviation 0.5.
  - $\tau = 0.05$ is the KL regularizer.
- Formulation:
  - $F_k(\pi) := \tau \text{KL}(\pi || \pi_{\text{ref}}) - \langle \pi, R_k \rangle = \tau[\sum_{a \in \mathcal{A}} \pi(a) \log \frac{\pi(a)}{\pi_{\text{ref}}(a)}] - \langle \pi, R_k \rangle$
  - $\nabla F_k(\pi) = \tau(\log \pi - \log \pi_{\text{ref}} + \vec{1}) - R_k$
  - $L_k := \frac{\tau}{\delta}$ on $\Delta_\delta := \{\pi \in \Delta_{|\mathcal{A}|} : \pi(a) \geq \delta\}$
  - $\mu_k := \tau$
- Plots and Tables:

5.3 MO-Gymnasium Benchmarks

5.3.1 Deep Sea Treasure (DST) Problem

- Setting: A grid world containing obstacle cells, treasure cells, and water which can be modeled as a MDP $\mathcal{M} := (\mathcal{S}, \mathcal{A}, \mathcal{P}, r, \gamma, \rho, \tau)$
  - $\mathcal{S}$: 72 non-obstacle cells
  - $\mathcal{A} = \{\text{up, down, left, right}\}$
  - $\mathcal{P}(s'|s, a)$ is the deterministic transition dynamics where the action either moves to the adjacent cell in the chosen direction or stays at the current cell if the chosen direction is outside the grid or an obstacle.
  - $r(s, a) = [r_1(s, a), r_2(s, a)]$ is the reward vector consists of
    - $-r_1(s, a) = 1$ is the time cost incurred at each step until a treasure is reached.
    - $r_2(s, a)$ is the treasure reward obtained upon reaching a treasure cell and is zero otherwise.
  - $\gamma = 0.999$ is the discount factor.
  - ρ is the initial state distribution.
  - $\tau = 1.5$ is the KL regularizer.
  - π_{ref} is the uniform policy.

- Formulation:
  - Goal: Find a policy $\pi \in \Delta_{|\mathcal{A}|}^{|\mathcal{S}|}$ minimizing two regularized discounted objectives.
  - $F_k(\pi) := \mathbb{E}_{\pi \in \Delta_{|\mathcal{A}|}^{|\mathcal{S}|}}[\sum_{t=0}^{\infty} \gamma^t (\tau \text{KL}(\pi(\cdot|s_t)||\pi_{\text{ref}}) - r_k(s_t, a_t))]$
- Metrics: CV, Gap Ratio, IGD instead of gradient norm?
- Plots and Tables:

5.3.2 Fishwood Problem

- Setting: A regularized MDP $\mathcal{M} := (\mathcal{S}, \mathcal{A}, \mathcal{P}, r, \gamma, \rho, \tau)$
  - $\mathcal{S} = \{0\text{:fishing state}, 1\text{:woods}\}$
  - $\mathcal{A} = \{0\text{:fishing}, 1\text{:collecting woods}\}$
  - $\mathcal{P}(s'|s, a)$ is the deterministic transition dynamics where action 0 leads to the fishing state and action 1 leads to the woods.
  - $r(s, a) = [r_1(s, a), r_2(s, a)]$ is the stochastic reward vector consists of
    - $r_1(s, a) = 1$ w.p. p_{wood}, if $s = 1$
    - $r_2(s, a) = 1$ w.p. p_{fish}, if $s = 0$
  - $\gamma = 0.995$ is the discount factor.
  - ρ is the initial state distribution.
  - $\tau = 0.5$ is the KL regularizer.
  - π_{ref} is the uniform policy.
- Formulation:
  - Goal: Find a policy $\pi \in \Delta_{|\mathcal{A}|}^{|\mathcal{S}|}$ minimizing two regularized discounted objectives.
  - $F_k(\pi) := \mathbb{E}_{\pi \in \Delta_{|\mathcal{A}|}^{|\mathcal{S}|}}[\sum_{t=0}^{\infty} \gamma^t (\tau \text{KL}(\pi(\cdot|s_t)||\pi_{\text{ref}}) - r_k(s_t, a_t))]$
- Plots and Tables:

5.3.3 MO-Mountaincar Problem

- Setting: An underpowered car must build momentum by moving back and forth in a valley in order to reach the goal near the top of the right hill. The agent is penalized at every time step and also incurs additional penalties for control behavior, inducing a trade-off between fast completion and action efficiency. It can be modeled as a multi-objective MDP $\mathcal{M} := (\mathcal{S}, \mathcal{A}, \mathcal{P}, r, \gamma, \rho, \tau)$
  - $\mathcal{S} = \{(x_t\text{: car position}, v_t\text{: velocity})\} \subseteq [-1.2, 0.6] \times [-0.07, 0.07]$
  - $\mathcal{A} = \{0\text{: accelerating to the left}, 1\text{: taking no push}, 2\text{: accelerating to the right}\}$
  - $\mathcal{P}(s'|s, a)$ is the deterministic transition dynamics where
    - $\tilde{a}_t = a_t - 1 \in \{-1, 0, 1\}$
    - $v_{t+1} = v_t + 0.001\tilde{a}_t - 0.0025\cos(3x_t)$
    - $x_{t+1} = x_t + v_{t+1}$
  - $r(s, a) = [r_1(s, a), r_2(s, a)]$ is the reward vector consists of
    - $r_1(s, a) = -1$
    - $r_2(s, a) = \begin{cases} -1, a_t = 2 \\ 0, \text{o.w.} \end{cases}$
  - $\gamma = 0.995$ is the discount factor.
  - ρ is the initial state distribution which is sampled uniformly from ([-0.6, -0.4], 0).
  - $\tau = 0.05$ is the KL regularizer.
  - π_{ref} is the uniform policy.
- Formulation:
  - Goal: Find a policy $\pi \in \Delta_{|\mathcal{A}|}^{|\mathcal{S}|}$ minimizing two regularized discounted objectives.
  - $F_k(\pi) := \mathbb{E}_{\pi \in \Delta_{|\mathcal{A}|}^{|\mathcal{S}|}}[\sum_{t=0}^{\infty} \gamma^t (\tau \text{KL}(\pi(\cdot|s_t)||\pi_{\text{ref}}) - r_k(s_t, a_t))]$
  - Inner solver: Use DQN trained with Adam and learning rate 10^{-3}.
- Plots and Tables:

5.3.4 MO-Fruit-Tree Problem

5.4 Stochastic LLM Multi-Objective Fine-tuning

- Setting: Bi-objective alignment for Reddit summarization using prompts from the openai/summarize_from_feedback dataset, with initial policy: Qwen/Qwen2.5-0.5B-Instruct. This is modeled as a contextual RL problem
  - $\mathcal{S} =$ Prompts
  - $\mathcal{A} =$ Generated Summary
  - $r(s,a) = [r_1(s,a), r_2(s,a)]$ is the reward vector consists of
    - Summarization-quality (Helpful) $r_1(s,a)$: Tristan/gpt2_reward_summarization
    - Factual (Harmless) $r_2(s,a)$: CogComp/bart-faithful-summary-detector
  - $\tau = 0.05$ is the KL regularizer.
  - π_{ref} is the uniform policy.
- Formulation:
  - Goal: Find a policy π maximizing two scalarized regularized reward.
  - $F_k(\pi) := \mathbb{E}_{s\sim D, a\sim\pi(\cdot|s)}[\tau \text{KL}(\pi(\cdot|s)||\pi_{\text{ref}}) - r_k(s,a)]$
- Training protocol:
  - Use PPO to solve each scalarized subproblem.
  - Learning rate: 5×10^{-5}
  - Batch size: 64
  - Mini-batch size: 16
  - Generated response length: [16, 32]
- Baseline: Soup
- Plots and Tables:

5.5 DPO loss weighting (DPOLW)

We consider the simplest formulation for multi-objective DPO loss, which weights each DPO loss function by a weight parameter λ.

Given dataset $D_k := \{(x^i, y_w^i, y_l^i)\}_{i=1}^{n_k}$, the corresponding DPO loss with respect to policy π_θ and reference policy π_{sft} is

$$\mathcal{L}_{\text{DPO}}(\pi_\theta, \pi_{\text{sft}}, D_k) := E_{(x,y_w,y_l)\sim D_k}[-\log\sigma(z(\theta))]$$

with sample average approximation $F_k(\theta) := -\frac{1}{n_k}\sum_{i=1}^{n_k}\log\sigma(z_i(\theta))$

where

$$z_i(\theta) := \beta[\log\frac{\pi_\theta(y_w^i|x^i)}{\pi_{\text{sft}}(y_w^i|x^i)} - \log\frac{\pi_\theta(y_l^i|x^i)}{\pi_{\text{sft}}(y_l^i|x^i)}]$$

In addition, $F_k(\theta)$ is non-convex due to the neural network structure of π_θ.

Hence, one simple multi-objective DPO formulation is the following DPO loss weighting (DPOLW) formulation:

$$\min_{\theta\in\Theta}\sum_{k=1}^{K}\lambda_k F_k(\theta)$$

for all $\lambda \in \Delta_K$.

This DPOLW formulation is a smooth approximation to the loss function $\mathcal{L}_{\text{MODPO}}$ presented in section 3.1 of the paper. We compare the Pareto front generated by solving the DPOLW formulation using the adaptive bundle method to the MODPO formulation presented in the paper in terms of accuracy and efficiency.

6 Conclusions and future directions

We proposed a first-order bundle method with convergence guarantee for smoothing the Pareto front of non-convex multi-objective optimization. In contrast to uniform discretization, which solves each scalarized subproblem independently, our method maintains a *bundle* of zeroth and first-order information of the objective functions and reuses it across the simplex. From this bundle it constructs a progress criterion that certifies worst-case stationarity *uniformly* over all weight vectors $\lambda \in \Delta_K$.

We established both iteration and oracle complexity bounds for the resulting algorithm under generic non-convexity (Theorem 1), and we demonstrated empirically that the adaptive method reaches the worst-case stationarity faster than the uniform discretization method at a substantially smaller computation and gradient-oracle budget.

Several directions remain open. First, our analysis targets the generic non-convex regime; sharper rates should be available under additional structure such as the strong convexity and Polyak–Łojasiewicz condition, which holds for many over-parameterized learning problems and would tighten the inner-loop oracle complexity.

Second, our adaptive idea could be applied to solving smooth Tchebycheff scalarization for multi-objective optimization, which has applications in tracing the solutions on the non-convex part of the optimal Pareto front [LZY+24].

Acknowledgements

References

[Bis06] C.M. Bishop. *Pattern recognition and machine learning*, volume 4. Springer New York, 2006.

[Dés12] Jean-Antoine Désidéri. Multiple-gradient descent algorithm (MGDA) for multiobjective optimization. *Comptes Rendus Mathématique*, 350(5–6):313–318, 2012.

[LLJ+21] Bo Liu, Xingchao Liu, Xiaojie Jin, Peter Stone, and Qiang Liu. Conflict-averse gradient descent for multi-task learning. In *Advances in Neural Information Processing Systems*, volume 34, 2021.

[LZY+24] Xi Lin, Xiaoyuan Zhang, Zhiyuan Yang, Fei Liu, Zhenkun Wang, and Qingfu Zhang. Smooth tchebycheff scalarization for multi-objective optimization. 2024.

[Nes18] Yurii Nesterov. *Lectures on Convex Optimization*. Springer Publishing Company, Incorporated, 2nd edition, 2018.

[NSA+22] Aviv Navon, Aviv Shamsian, Idan Achituve, Haggai Maron, Kenji Kawaguchi, Gal Chechik, and Ethan Fetaya. Multi-task learning as a bargaining game. In *Proceedings of the 39th International Conference on Machine Learning*, volume 162 of *Proceedings of Machine Learning Research*, pages 16428–16446. PMLR, 2022.

[R97] CARUANA R. Multitask learning : A knowledge-based source of inductive bias. *Machine Learning*, 28:41–75, 1997.

[SK18] Ozan Sener and Vladlen Koltun. Multi-task learning as multi-objective optimization. 31, 2018.

[YBHNL26] Hoyeol Yoon, Seoungbin Bae, Nam Ho-Nguyen, and Dabeen Lee. Chebyshev center-based direction selection for multi-objective optimization and training pinns. 2026.

A Omitted Proofs

A.1 Proof of Proposition 1

Proof. For gradient descent on an L_λ-smooth (possibly nonconvex) function, Nesterov's descent analysis in Section 1.2.3 yields

$$\min_{\theta_i \in \mathcal{B}_m} \|\nabla_\lambda F(\theta_i)\|^2 \leq \frac{1}{m}\left[\frac{L_\lambda}{\omega}\big(F_\lambda(\theta_1) - F_\lambda^*\big)\right],$$

see [Nes18], eqs. (1.2.22). By picking gradient descent step size $\frac{1}{L_\lambda}$, we have $\omega := \frac{1}{2}$.

Hence,

$$\mathrm{GN}(\lambda; \mathcal{B}_m) = \min_{\theta_i \in \mathcal{B}_m} \|\nabla F_\lambda(\theta_i)\|^2 \leq \frac{2L_\lambda[F_\lambda(\theta_1) - F_\lambda^*]}{m}.$$

Let $C_\lambda > 0$ be such that $2[F_\lambda(\theta_1) - F_\lambda^*] \leq C_\lambda$. □

A.2 Proof of Theorem 1

Proof. Consider an $\frac{\epsilon}{3\mathrm{LipGN}}$-net of Δ_K w.r.t. the ℓ_1-norm. That is, a finite set of points $N \subseteq \Delta_K$ such that, for all $\lambda \in \Delta_K$, there exists $\hat{\lambda} \in N$ with

$$\|\lambda - \hat{\lambda}\|_1 \leq \frac{\epsilon}{3\mathrm{LipGN}}.$$

We have for all $t \geq 1$ and for all $\lambda \in \Delta_K$, there exists $\hat{\lambda} \in N$ such that

$$\mathrm{GN}(\lambda; \mathcal{B}_t) \leq \mathrm{GN}(\hat{\lambda}; \mathcal{B}_t) + \frac{\epsilon}{3}.$$

Therefore, it suffices to show that $\sup_{\hat{\lambda} \in N} \mathrm{GN}(\hat{\lambda}; \mathcal{B}_t) \leq \frac{2\epsilon}{3}$ at some iteration t.

At any iteration t of Algorithm 2, let C_{λ_t} be the constant in Corollary 4.1 such that

$$\mathrm{GN}(\lambda_t; \mathcal{B}_{t+1}) < \frac{\epsilon}{3}$$

for $M_t = \lceil \frac{3C_{\lambda_t} L_{\lambda_t}}{\epsilon} \rceil$.

Now, let $\hat{\lambda}_t \in N$ be the projection onto the net N for λ_t.

$$\mathrm{GN}(\hat{\lambda}_t; \mathcal{B}_{t+1}) \leq \mathrm{GN}(\lambda_t; \mathcal{B}_{t+1}) + \frac{\epsilon}{3} \leq \frac{2\epsilon}{3},$$

by combining with the previous inequality.

Thus, at every iteration t of Algorithm 2, we can examine the list of values $\{\mathrm{GN}(\hat{\lambda}; \mathcal{B}_t)$ for $\hat{\lambda} \in N\}$. Note that these values are all monotone decreasing by the global monotonicity property. At every iteration, we just demonstrated that at least one of these values becomes less than or equal to $\frac{2\epsilon}{3}$ and therefore remains less than or equal to $\frac{2\epsilon}{3}$ for all subsequent iterations. Thus, the total number of iterations before it is guaranteed that $\sup_{\hat{\lambda} \in N} \mathrm{GN}(\hat{\lambda}; \mathcal{B}_t) \leq \frac{2\epsilon}{3}$ is bounded by the size of N. As mentioned before, $\sup_{\hat{\lambda} \in N} \mathrm{GN}(\hat{\lambda}; \mathcal{B}_t) \leq \frac{2\epsilon}{3}$ ensures that $\mathrm{GN}_t^* \leq \epsilon$. Note that the size of N can be bounded by $\left(\frac{C\mathrm{LipGN}}{\epsilon}\right)^{K-1}$. As for the total oracle calls, by Corollary 4.1, the number of oracle calls at each inner iteration to ensure $\mathrm{GN}(\lambda_t; \mathcal{B}_{t+1}) < \frac{\epsilon}{3}$ is $\frac{E}{\epsilon}$.

This completes the proof. □

B Implementation Details

B.1 Calibrating Uniform Discretization

B.2 An adaptive algorithm

C Additional Figures